

Question Bank for CIE- I

Unit- II

1. Explain concept of verification and validation
2. Explain levels of testing
3. Explain Testing Life cycle in detail.
4. Write note on - Test-Driven Development(TDD), Behavior-Driven Development(BDD), and Acceptance Test-Driven Development(ATDD)
5. Develop test cases for ATM system/online shopping/flight booking system/Railway reservation system
6. Construct test plan for ATM system/online shopping/flight booking system/Railway Reservation System
7. Explain Bug life cycle
8. What is defect repository? Explain with suitable example.
9. Explain Testing Life cycle in detail.
10. Develop test case for Username and Password

Explain concept of verification and validation

Verification and Validation (V&V) are two important concepts in software engineering and quality assurance used to ensure that a software system meets its requirements and performs as expected.

1. Verification – “*Are we building the product right?*”

Definition:

Verification is the process of evaluating software during development to ensure that it meets the specified requirements and design specifications.

Key Features:

- Focuses on *process* and *intermediate work products* (like design documents, source code, etc.)

- . Ensures that the software is being built *correctly* according to requirements.
- . Conducted through reviews, inspections, walkthroughs, and desk-checking.
- . Does **not** involve executing the code.

Examples of Verification:

- . Requirements review
- . Design review
- . Code inspection
- . Static analysis

2. Validation – “*Are we building the right product?*”

Definition:

Validation is the process of evaluating the final software product to ensure that it meets the **business needs and user expectations**.

Key Features:

- . Focuses on the *actual product*.

- . Ensures that the right product has been built for the customer.
- . Involves executing the software and checking if it behaves as expected.
- . Performed through **testing** (unit testing, system testing, user acceptance testing, etc.)

Examples of Validation:

- . System testing
- . Integration testing
- . User Acceptance Testing (UAT)
- . Beta testing

Comparison Table:

Aspect	Verification	Validation
Purpose	Are we building the product right?	Are we building the right product?
Focus	Process-oriented	Product-oriented

Activities	Reviews, walkthroughs, inspections	Testing and execution
Timing	During development	After implementation
Performed by	Developers, QA, project team	Testers, end users, QA
Execution of code	Not required	Required

Summary:

- . **Verification** ensures that the product is developed correctly.
- . **Validation** ensures that the correct product is developed.

Both are essential for delivering high-quality, reliable software.

Explain levels of testing

Levels of Testing in Software Engineering

Software testing is performed in **multiple levels** to ensure that errors are identified early and the software works correctly as a whole.

Each level focuses on a different aspect of the system.

◆ 1. Unit Testing

Definition:

Testing individual components or modules of a program in isolation to ensure they work as expected.

Focus:

- . Smallest testable parts like functions, classes, or methods.

Who performs it?

- . Developers

Tools:

- . JUnit (Java), NUnit (.NET), PyTest (Python), etc.

Example:

Testing a function that calculates the sum of two numbers.

♦ 2. Integration Testing

Definition:

Testing the interaction between two or more integrated units or modules.

Focus:

- . Data flow and interfaces between modules.

0

Who performs it?

- . Developers or testers

Approaches:

- . Top-down
- . Bottom-up
- . Big bang
- . Sandwich

Example:

Testing the login module interacting with the user authentication system.

3. System Testing

Definition:

Testing the complete and integrated software

system to ensure it meets the specified requirements.

Focus:

- . End-to-end testing of the whole application.

Who performs it?

- . Independent testing team

Types:

- . Functional testing
- . Non-functional testing (performance, security, etc.)

Example:

Testing an entire e-commerce website from login to payment checkout.

4. Acceptance Testing

Definition:

Testing to validate the software against user needs and business requirements.

Focus:

- . Final validation before the software goes live.

Who performs it?

- . End users or clients

Types:

- . Alpha Testing (in development environment)
- . Beta Testing (in real-world environment)

Example:

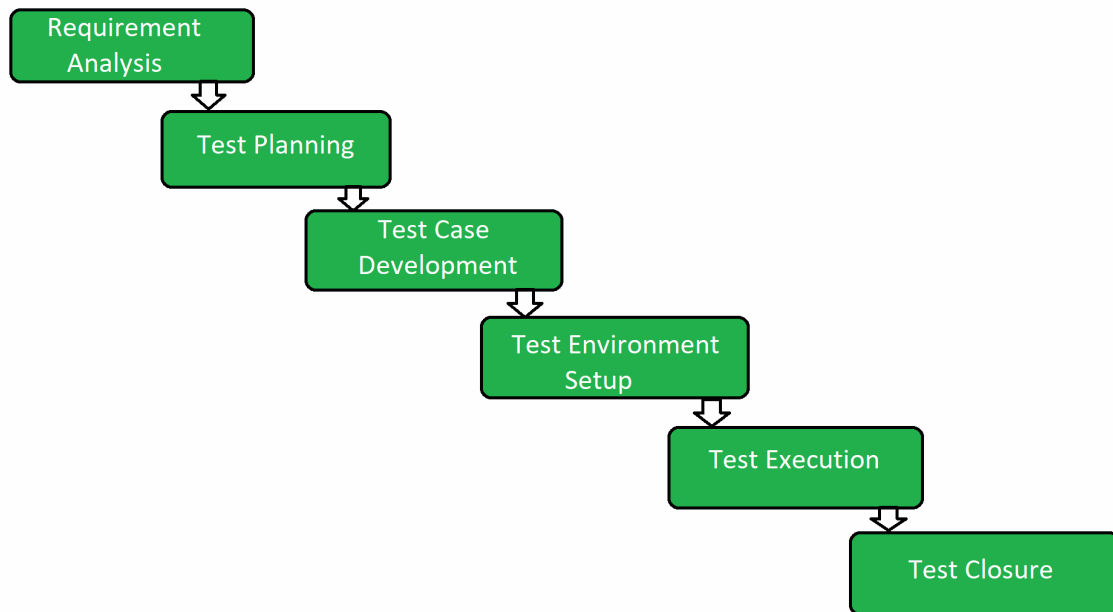
A client tests an inventory management system before deploying it in the warehouse.

Summary Table:

Level	Focus	Performed By	Purpose
Unit Testing	Individual modules/functions	Developers	Ensure each unit works correctly
Integration Testing	Interfaces between units	Developers/Testers	Ensure combined units work together properly
System Testing	Complete system	Independent testers	Ensure system meets functional

			and non-functional requirements
Acceptance Testing	User/business requirements	End-users/Clients	Ensure the system is ready for release

1. Explain Testing Life cycle in detail.

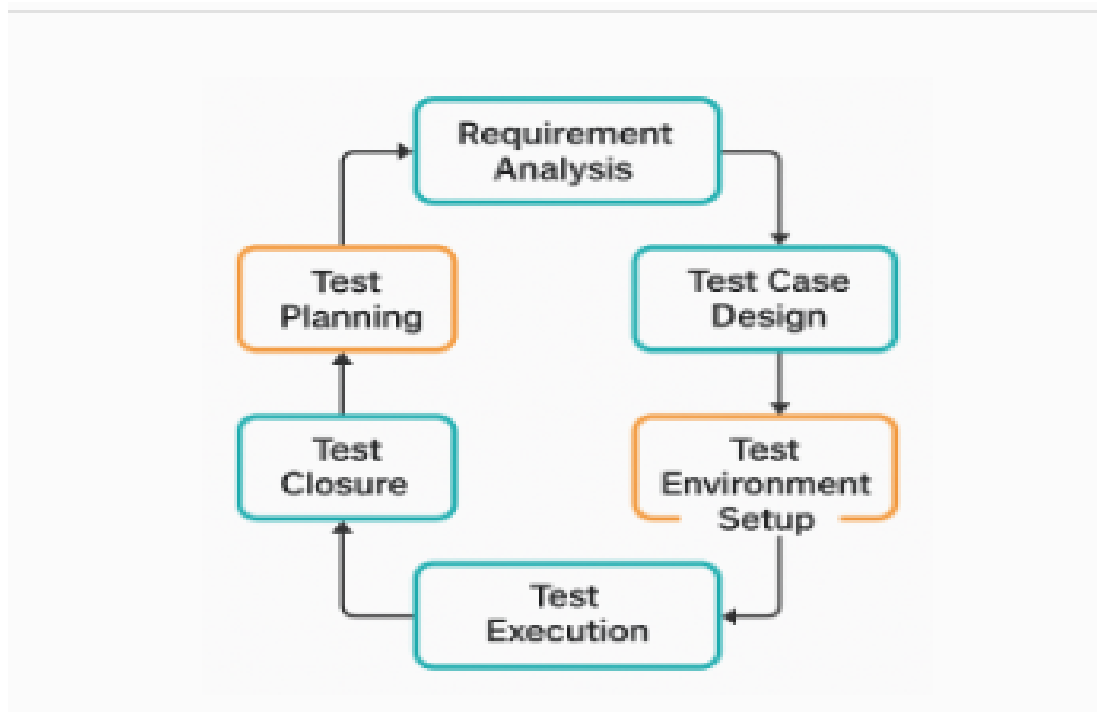


Key Phases of STLC:

- . **Requirement Analysis:** Understanding the testing requirements, including functional and non-functional aspects.
- . **Test Planning:** Defining the overall testing strategy, scope, resources, and timelines.
- . **Test Case Development:** Creating detailed test cases to cover various scenarios and functionalities.
- . **Environment Setup:** Configuring the necessary hardware and software environment for testing.
- . **Test Execution:** Running the test cases and logging the results.

- **Test Cycle Closure:** Analyzing test results, reporting defects, and preparing a test closure report.
-

Software Testing Life Cycle (STLC) – Detailed Explanation



The **Software Testing Life Cycle (STLC)** is a sequence of specific activities conducted during the testing process to ensure software quality goals are met. Each phase has defined objectives and deliverables, ensuring a structured and efficient approach to software testing.

Phases of the Testing Life Cycle:

1. Requirement Analysis

Objective:

Understand and analyze the testing requirements based on software requirement specifications (SRS), business requirements, or use cases.

Activities:

- . Identify testable requirements.
- . Communicate with stakeholders to clarify doubts.
- . Determine types of tests (functional, performance, security, etc.)

Deliverables:

- . Requirement Traceability Matrix (RTM)
- . Automation feasibility report (if applicable)

2. Test Planning

Objective:

Define the scope, objectives, approach, resources, and schedule for testing activities.

Activities:

- . Create a test strategy and test plan.
- . Estimate effort and time.
- . Define roles and responsibilities.
- . Identify testing tools and environment needs.

Deliverables:

- . Test Plan document
- . Effort estimation
- . Risk and mitigation plan

3. Test Case Design / Test Development

Objective:

Design detailed test cases and test scripts covering all functional and non-functional requirements.

Activities:

- . Write test cases with input data and expected results.
- . Design test data.
- . Review and get approval for test cases.

Deliverables:

- . Test cases/scripts
- . Test data
- . Updated RTM

4. Test Environment Setup

Objective:

Prepare the hardware and software environment in which tests will be executed.

Activities:

- . Install and configure software, database, servers.
- . Set up network, user accounts, access rights.
- . Validate the test environment.

Deliverables:

- . Environment setup checklist
- . Smoke test result to confirm readiness

5. Test Execution

Objective:

Run the test cases and compare actual vs. expected results.

Activities:

- . Execute manual or automated test cases.
- . Log defects in defect tracking tool.
- . Retest and perform regression testing.

Deliverables:

- . Test execution report
- . Defect reports
- . Updated RTM

6. Test Closure

Objective:

Summarize and evaluate testing efforts and results to confirm completion.

Activities:

- . Finalize test reporting.
- . Document lessons learned.
- . Archive test artifacts for future use.
- . Review test deliverables.

Deliverables:

- . Test summary report
- . Defect closure report
- . Test metrics
- . Test closure document

Summary Table:

Phase	Key Activities	Key Deliverables
Requirement Analysis	Analyze and identify test requirements	RTM, feasibility report

Test Planning	Define scope, tools, schedule, and responsibilities	Test Plan, estimation, risk plan
Test Case Design	Write and review test cases and test data	Test cases, test data, updated RTM
Test Environment Setup	Prepare environment and validate configuration	Environment setup checklist, smoke test
Test Execution	Execute tests, log defects, retest, regression testing	Test execution report, defect log
Test Closure	Analyze test results, report	Test summary

	metrics, document learnings	report, test closure report
--	-----------------------------------	-----------------------------------

Importance of STLC:

- . Ensures systematic and consistent testing
- . Helps identify defects early
- . Improves software quality
- . Reduces cost and time of defect fixing

Write note on - Test-Driven Development(TDD), Behavior-Driven Development(BDD), and Acceptance Test-Driven Development(ATDD)

1. Test-Driven Development (TDD)

Definition:

TDD is a software development approach where **tests are written before the code**.

The idea is to write a failing test case first, then write code to pass the test, and finally refactor the code.

Process (Red-Green-Refactor cycle):

1. **Red** – Write a failing test case.
2. **Green** – Write minimal code to pass the test.
3. **Refactor** – Improve the code while ensuring tests still pass.

Benefits:

- . Ensures better test coverage.
- . Encourages modular and flexible code.
- . Bugs are caught early.

Tools:

JUnit (Java), NUnit (.NET), PyTest (Python)

2. Behavior-Driven Development (BDD)

Definition:

BDD is an extension of TDD that emphasizes **collaboration between developers, testers, and business stakeholders**. It describes the system's behavior using **natural language**.

Process:

- . Define behavior in a "Given-When-Then" format (using Gherkin language).
- . Convert scenarios into automated tests.
- . Write code that satisfies the defined behavior.

Example:

pgsql

CopyEdit

Given a logged-in user

When they add an item to the cart

Then the item should appear in the cart

Benefits:

- . Improves communication between teams.
- . Provides clear documentation.
- . Focuses on business value.

Tools:

Cucumber, SpecFlow, Behave

3. Acceptance Test-Driven Development (ATDD)

Definition:

ATDD involves **writing acceptance tests before development begins**, based on **customer and business requirements**. All stakeholders collaborate to define the expected behavior.

Process:

- . Discuss and agree on acceptance criteria.
- . Write acceptance tests.
- . Develop the system to pass those tests.

Focus:

- . Meeting business needs and **user acceptance**.

Benefits:

- . Reduces misunderstandings.
- . Ensures the system aligns with user expectations.

- . Enhances test coverage from the user's point of view.

Tools:

FitNesse, Robot Framework, Cucumber

Summary Table:

Approach	Written By	Written Before	Focus Area	Tools
TDD	Developers	Development	Code correctness	JUnit, PyTest
BDD	Developers + QA + BA	Development	Behavior from user view	Cucumber
ATDD	Developers + Testers + Customer	Development	Acceptance criteria	FitNesse, Cucumber

	rs			
--	----	--	--	--

1. Develop test cases for ATM system/online shopping/flight booking system/Railway reservation system

Online Shopping System – Sample Test Cases

Test Case ID	Test Scenario / Feature	Test Steps	Expected Result
SHOP_TC_01	User Registration	Fill registration form with valid data and submit	Account is successfully created and confirmation message shown

SHOP_TC_02	Registration with missing fields	Leave mandatory field (e.g., email) blank and submit	Validation error shown: "Email is required"
SHOP_TC_03	Login with valid credentials	Enter correct username and password	User is redirected to the home/dashboard page
SHOP_TC_04	Login with invalid password	Enter valid username and wrong password	Error message shown: "Invalid credentials"

		d	
SHOP_TC_05	Product Search	Type product name in search bar and click search	List of relevant products is displayed
SHOP_TC_06	Add product to cart	Click "Add to Cart" on a displayed product	Product is added to cart and cart count is updated
SHOP_TC_07	View Cart	Click on "Cart" icon	Cart page shows all added products
SHOP_TC	Remove	Click	Product is

_08	product from cart	"Remove" on a product in cart	removed, cart is updated
SHOP_TC_09	Apply valid coupon code	Enter valid promo code during checkout	Discount is applied to the total amount
SHOP_TC_10	Apply invalid coupon code	Enter expired or incorrect promo code	Error message shown: "Invalid or expired coupon code"
SHOP_TC_11	Checkout with valid	Fill delivery	Order placed

	details	address, select payment, confirm order	successfully with order confirmation message
SHOP_TC_12	Checkout with missing delivery address	Leave delivery address blank during checkout	Validation error shown: "Address is required"
SHOP_TC_13	Payment with invalid card	Enter incorrect card number or expiry date	Payment fails and error message is shown

SHOP_TC_14	Payment with valid card	Enter valid card details and proceed	Payment is successful, redirected to order confirmation
SHOP_TC_15	View Order History	Login → Navigate to "My Orders"	All past orders are listed with status
SHOP_TC_16	Cancel an Order	Go to "My Orders" → Select order → Click "Cancel"	Order is cancelled and status updated to "Cancelled"

SHOP_TC_17	Logout Functionality	Click on "Logout" after login	User is logged out and redirected to the login page
-------------------	----------------------	-------------------------------	---

Optional Enhancements:

You can expand these with:

- . **Preconditions** (e.g., user must be logged in)
- . **Test Data**
- . **Expected vs Actual Result**
- . **Priority/Severity Tags**

Let me know if you'd like this in **Excel**, **PDF**, or as a **test plan template**.

module-wise breakdown of test cases for an **Online Shopping System**, organized into key modules:

1. User Registration & Login Module

Test Case ID	Test Scenario	Test Steps	Expected Result
REG_TC_01	Valid user registration	Enter valid Name, Email, Password, Phone → Submit	"Registration successful" message appears
REG_TC_02	Registration with missing fields	Leave Email blank → Submit	Validation error: "Email is required"
REG_TC_03	Registration with invalid	Enter abc@xyz	Validation error: "Enter a valid email"

	email format	without domain → Submit	address"
REG_TC _04	Login with correct credential s	Enter correct Email and Passwo rd → Login	Redirect to homepage/dash board
REG_TC _05	Login with incorrect passwor d	Enter register ed Email and wrong passwo rd → Login	"Invalid credentials" error message

REG_TC_06	Password recovery	Click "Forgot Password" → Enter Email	Password reset link sent to registered email
-----------	-------------------	---------------------------------------	--

2. Product Browsing and Search Module

Test Case ID	Test Scenario	Test Steps	Expected Result
BROWSE_TC_01	View product categories	Click on "Electronics", "Clothing", etc.	Products under selected category displayed
BROWSE_TC_02	Search for a product	Enter "Laptop" in search	Matching laptop products

		bar → Click search	displaye d
BROWSE_TC _03	Search with no matchin g result	Enter random string like "xyz123" → Search	Message: "No products found"
BROWSE_TC _04	View product details	Click on a product	Detailed product page with descripti on, price shown
BROWSE_TC _05	Sort product s by price (low to	Click "Sort by: Price – Low to High"	Products sorted accordin gly

	high)		
--	-------	--	--

3. Shopping Cart Module

Test Case ID	Test Scenario	Test Steps	Expected Result
CART_TC_01	Add product to cart	On product page → Click "Add to Cart"	Product added, cart count increases
CART_TC_02	View cart	Click on "Cart" icon	List of added products shown
CART_TC_03	Change product quantity	In cart → Increase quantity to 2	Total amount updates correctly

CART_TC_04	Remove product from cart	Click "Remove" next to a product	Product removed, total updates
CART_TC_05	Cart with no products	Remove all items	Message: "Your cart is empty"

4. Wishlist Module

Test Case ID	Test Scenario	Test Steps	Expected Result
WISH_TC_01	Add item to wishlist	On product page → Click "Add to Wishlist"	Product added to wishlist

WISH_TC_02	View wishlist	Go to "Wishlist" section	All saved items are shown
WISH_TC_03	Move product from wishlist to cart	Click "Move to Cart"	Item removed from wishlist and added to cart

5. Checkout and Payment Module

Test Case ID	Test Scenario	Test Steps	Expected Result
PAY_TC_01	Proceed to checkout	Click "Checkout" from cart	Redirect to address & payment screen
PAY_TC_02	Enter valid shipping	Fill in address, city, pin	Address saved successfull

	address	code → Continue	y
PAY_TC_03	Apply valid coupon	Enter valid coupon code → Apply	Discount is reflected in total price
PAY_TC_04	Apply invalid coupon	Enter expired or wrong code → Apply	Error: "Invalid coupon code"
PAY_TC_05	Pay with valid credit/debit card	Enter correct card details → Confirm	Payment succeeds, order placed
PAY_TC_06	Pay with invalid card	Enter fake card number or	Payment fails with appropriat

		expired card	e message
--	--	--------------	-----------

6. Order Management Module

Test Case ID	Test Scenario	Test Steps	Expected Result
ORDER_TC_01	View order confirmation	After successful payment → Show confirmation	Order ID, summary, delivery estimate shown
ORDER_TC_02	View past orders	Go to "My Orders"	List of past orders with statuses shown
ORDER_TC_03	Cancel pending	Click "Cancel"	Status updated

	order	on an unpaid or processing order	to "Cancelled"
ORDER_TC_04	View order invoice	Click "Invoice" for a completed order	Invoice PDF/HTML shown

7. User Profile Module

Test Case ID	Test Scenario	Test Steps	Expected Result
PROFILE_TC_01	View and edit profile	Login → Go to "My Profile" → Edit name/pho	Profile updated successfully

		ne	
PROFILE_TC_02	Change password	Old password + new password → Submit	Password changed confirmation shown
PROFILE_TC_03	Save default shipping address	Enter and save address in profile settings	Address saved and used in future orders

8. Logout and Session Handling

Test Case ID	Test Scenario	Test Steps	Expected Result
SESSION_TC_01	Logout	Click on "Logout	Redirected to login page

		"	
SESSION_TC_02	Session timeout	Stay idle for 15+ minutes	Auto logout with session timeout message

Flight reservation system

1. Flight Search Module

Test Case ID	Test Scenario	Test Steps / Input	Expected Result
FS_TC_01	Search valid flight	Source: Pune,	Show list of

		Destination: Delhi, Date: 15/08/2025	available flights
FS_TC_02	Invalid source/destination	Source: Abcxyz	Show error: “Invalid city name”
FS_TC_03	Empty search fields	Click search with all fields blank	Show validation message
FS_TC_04	Search for past date	Date: 01/01/2024	Show error: “Invalid travel date”

2. Passenger Details Module

Test Case ID	Test Scenario	Test Steps / Input	Expected Result
PD_TC_01	Valid passenger information	Fill: Name, Gender, Age, ID	Proceeds to payment
PD_TC_02	Missing required fields	Leave name or ID blank	Show validation message
PD_TC_03	Multiple passengers entry	Add 3 passengers with correct details	All passengers added successfully

3. Payment Module

Test Case ID	Test Scenario	Test Steps / Input	Expected Result
---------------------	----------------------	---------------------------	------------------------

PM_TC_01	Valid credit card payment	Card No: valid, CVV, Expiry	Payment success, ticket issued
PM_TC_02	Invalid card details	Wrong card no./expired date	Show error: Payment failed
PM_TC_03	Cancel during payment	Click "Cancel" on payment screen	Booking not confirmed

4. Ticket Confirmation Module

Test Case ID	Test Scenario	Test Steps / Input	Expected Result
TC_TC_01	Check booking	Book a flight, go	Show ticket and

	confirmatio n	to “My Bookings ”	booking details
TC_TC_0 2	Email/SMS notification	Book ticket, check inbox	Email/SM S with ticket sent

5. Ticket Cancellation Module

Test Case ID	Test Scenario	Test Steps / Input	Expected Result
CN_TC_0 1	Cancel ticket before departur e	Click "Cancel" under booking before flight time	Show confirmatio n and refund status
CN_TC_0	Cancel	Try	Error:

2	expired ticket	cancellin g after flight time	Cancellation not allowed
---	----------------	--	-----------------------------

6. Admin Panel Module

Test Case ID	Test Scenario	Test Steps / Input	Expected Result
AP_TC_01	Admin login	Username: admin, Password: pass123	Dashboard opens
AP_TC_02	Add new flight	Fill flight details → Save	Flight appears in search
AP_TC_03	Edit flight schedule	Update departure time or price	Changes reflected for users

7. Security Module

Test Case ID	Test Scenario	Test Steps / Input	Expected Result
SC_TC_01	SQL injection attempt in input field	Input: ' OR 1=1 -- in search field	Input rejected, app remains secure
SC_TC_02	Unauthorized admin access	Try opening admin page without login	Redirect to login page or access denied

Test plan

Test Plan for Flight Booking System (Applicable to other systems too)

1. Test Plan ID:

TP-FBS-001

2. Project Title:

Flight Booking System

3. Objective:

To verify the functionality, usability, performance, and security of the Flight Booking System and ensure it meets the specified requirements.

4. Scope of Testing:

In Scope:

- . Flight search and selection

- . Passenger data input
- . Booking and payment process
- . Ticket confirmation and email
- . Booking cancellation and refund
- . Admin panel functions

Out of Scope:

- . Backend airline system API testing (if outsourced)
- . Testing third-party payment gateway internals

5. Test Items:

- . Flight Search Function
- . Flight Selection
- . Passenger Data Entry
- . Payment Integration
- . Ticket Generation
- . Booking History
- . Admin Module (Add/Edit/Remove Flights)
- . Email/SMS Notifications

. Cancellation and Refund

6. Features to Be Tested:

Feature	Description
Search Flights	Based on source, destination, and date
Booking	Select flight, input details, and confirm
Payment	Process using valid and invalid cards
Confirmation	Generate ticket and notify customer
Admin Module	Manage flight schedule and fares
Security	Authentication and input validation

7. Features Not to Be Tested:

- . Backend airline API logic
- . Third-party SMS/email delivery systems
- . Non-functional requirements outside QA scope (like UI design consistency unless part of UX testing)

8. Test Approach:

- . **Functional Testing:** Validate all business rules and workflows
- . **System Testing:** End-to-end test of the booking journey
- . **Integration Testing:** Check interfaces between modules
- . **Usability Testing:** Verify the system is user-friendly
- . **Security Testing:** SQL injection, login access control
- . **Regression Testing:** Re-test after bug fixes or updates

9. Entry Criteria:

- . Requirements are finalized and approved
- . Test environment is ready
- . Test cases are prepared and reviewed
- . Unit testing is completed

10. Exit Criteria:

- . All critical test cases passed
- . No high or critical severity defects open
- . Test summary report reviewed and accepted
- . Sign-off from QA lead/client

11. Test Deliverables:

- . Test Plan
- . Test Cases and Test Scripts
- . Defect Log
- . Daily/Weekly Test Reports
- . Final Test Summary Report
- . Sign-off Document

12. Testing Schedule:

Activity	Start Date	End Date
Test Planning	01-Aug-2025	03-Aug-2025
Test Case Writing	03-Aug-2025	06-Aug-2025
Test Execution	07-Aug-2025	14-Aug-2025
Reporting & Closure	15-Aug-2025	16-Aug-2025

13. Environment Needs:

- . Web browser (Chrome, Firefox)
- . Database: MySQL/PostgreSQL
- . Application server
- . Internet connection
- . Admin and test user credentials

14. Roles and Responsibilities:

Role	Responsibility
QA Lead	Overall test planning and monitoring
QA Tester	Write and execute test cases
Developer	Fix defects and support testing
Client/Stakeholder	Review test outcomes and sign off

15. Risks and Mitigation:

Risk	Mitigation
Delay in requirement finalization	Buffer added to schedule
Unstable test environment	Coordinate with DevOps for backups
Third-party services are down	Use stubs/mocks for testing

16. Approval:

Name	Role	Signature
[QA Lead]	QA Lead	
[Project Manager]	PM	
[Client/Stakeholder]	Product Owner	

TEST PLAN FOR ATM SYSTEM

1. Test Plan ID

TP-ATM-001

2. Project Title

Automated Teller Machine (ATM) System

3. Objective

To verify that the ATM software functions correctly, securely, and reliably across all supported transactions, and meets both functional and non-functional requirements.

4. Scope of Testing

In Scope:

- . Card insertion and PIN validation
- . Balance inquiry
- . Cash withdrawal and deposit
- . Mini statement
- . Fund transfer
- . Session timeout
- . Receipt printing

Out of Scope:

- . ATM hardware testing (card reader, cash dispenser)
- . Bank core backend system (handled by integration team)

5. Test Items

- . Login/Authentication module
- . Transaction processing module (withdrawal, deposit, balance)
- . Funds transfer module
- . Mini statement module
- . Receipt generation module
- . Error handling module
- . Session timeout handling
- . Security features (PIN retry, logout)

6. Features to Be Tested

Feature	Description
Card Authentication	Card detection and PIN validation
Balance Inquiry	View account balance
Cash Withdrawal	Withdraw money within balance &

	daily limits
Deposit	Deposit into account (simulated)
Mini Statement	View last 5–10 transactions
Fund Transfer	Transfer to another account
Receipt Generation	Print/display transaction confirmation
Session Timeout	Auto logout after inactivity
PIN Locking	Lock card after 3 invalid attempts

7. Features Not to Be Tested

- . Hardware interactions (cash dispenser malfunction, card eject mechanism)

- . Integration with real-time banking servers
- . ATM network failover features

8. Test Approach

- . **Functional Testing:** Core ATM operations tested as black-box scenarios
- . **System Testing:** End-to-end flow from login to logout
- . **Regression Testing:** After bug fixes or updates
- . **Security Testing:** PIN validation, retry count, lockouts
- . **UI Testing:** ATM screen navigation and prompts
- . **Negative Testing:** Invalid inputs and edge cases

9. Entry Criteria

- . All functional requirements finalized and signed off

- . Test cases prepared and reviewed
- . Test environment set up (ATM simulator or interface)
- . Build deployed and stable

10. Exit Criteria

- . All critical test cases passed
- . No critical or high severity bugs open
- . Test summary reviewed and accepted
- . Sign-off received from QA and project manager

11. Test Deliverables

- . Test Plan
- . Test Cases
- . Test Scripts (if automation is used)
- . Bug Reports / Defect Log
- . Test Summary Report
- . Sign-off Document

12. Testing Schedule

Phase	Start Date	End Date
Test Planning	01-Aug-2025	03-Aug-2025
Test Design	04-Aug-2025	06-Aug-2025
Test Execution	07-Aug-2025	10-Aug-2025
Reporting	11-Aug-2025	12-Aug-2025

13. Environment Requirements

- . ATM simulation software or interface
- . Database server (to simulate user accounts)
- . Test card data with valid/invalid scenarios
- . Printer simulator (for receipt testing)

14. Roles and Responsibilities

Role	Responsibility
QA Lead	Prepare test plan, monitor execution
Testers	Create and execute test cases
Developers	Fix bugs and support testing
Project Manager	Schedule and coordinate

15. Risks and Mitigation

Risk	Mitigation
Hardware unavailability	Use simulation/testing tools
Late requirement changes	Freeze scope before test design
Network/API failure	Use mocks or simulated responses

16. Approval

Name	Role	Signature
[Your Name]	QA Lead	
[Manager Name]	Project Manager	
[Client/Stakeholder]	Product Owner	

TEST PLAN FOR ONLINE SHOPPING SYSTEM

1. Test Plan ID:

TP-OSS-001

2. Project Title:

Online Shopping System

3. Objective:

To verify the complete functionality, usability, and security of the Online Shopping System, ensuring it meets the specified requirements and provides a smooth user experience for customers and administrators.

4. Scope of Testing:

In Scope:

- . User registration/login
- . Product browsing and search
- . Add to cart and checkout
- . Payment gateway integration
- . Order history and invoice
- . Admin panel (product/category/order management)

Out of Scope:

- . Backend third-party logistics API

- . Live payment gateway processing (can be simulated)

5. Test Items:

- . Registration/Login module
- . Product catalog and search
- . Cart management
- . Checkout and payment
- . Order summary and history
- . Admin dashboard
- . Notifications (email/SMS)

6. Features to Be Tested:

Feature	Description
User Registration/Login	New user sign-up, authentication, password reset
Product Browsing/Search	Search/filter products by category/price/brand

Add to Cart	Add/update/remove items in cart
Checkout Process	Shipping address, billing info, apply discount
Payment Integration	Pay using cards, wallets, UPI (test mode)
Order Management	View, cancel, return orders
Admin Controls	CRUD operations for product/category/order
Notifications	Order confirmation emails and alerts

7. Features Not to Be Tested:

- . Physical delivery and logistics
- . Internal functionality of third-party payment gateways
- . Performance under massive load (unless performance testing is included)

8. Test Approach:

- . **Functional Testing:** Validate all user and admin workflows
- . **UI Testing:** Check layout consistency, responsiveness
- . **Security Testing:** SQL injection, password encryption
- . **Integration Testing:** Between modules like cart → checkout → order
- . **Regression Testing:** Post bug-fix or feature updates
- . **Negative Testing:** Invalid inputs, missing fields

9. Entry Criteria:

- . All features implemented and unit tested
- . Stable build available
- . Test environment is ready
- . Test cases reviewed and approved

10. Exit Criteria:

- . All critical test cases executed and passed
- . All major defects fixed and retested
- . Test summary submitted and accepted
- . UAT feedback incorporated

11. Test Deliverables:

- . Test Plan document
- . Test Scenarios & Test Cases
- . Test Data sets
- . Bug/Defect Report
- . Test Execution Report
- . Final Test Summary Report
- . QA Sign-off

12. Testing Schedule:

Activity	Start Date	End Date
Test Planning	01-Aug-	02-Aug-

	2025	2025
Test Case Design	03-Aug-2025	05-Aug-2025
Test Execution	06-Aug-2025	10-Aug-2025
Reporting & Review	11-Aug-2025	12-Aug-2025

13. Test Environment:

- . Web browser (Chrome, Firefox, Safari)
- . Backend database (e.g., MySQL/PostgreSQL)
- . Test server with deployed application
- . Mock payment gateway credentials

14. Roles and Responsibilities:

Role	Responsibility
QA Lead	Create test plan, monitor test progress

QA Tester	Write and execute test cases
Developer	Fix reported bugs
Product Owner	Review test outcomes and sign off

15. Risks and Mitigation:

Risk	Mitigation
Unstable build	Use version control and stable release
Delayed requirement finalization	Freeze scope before test case writing
Integration failure with payment API	Use sandbox/test mode with mock data

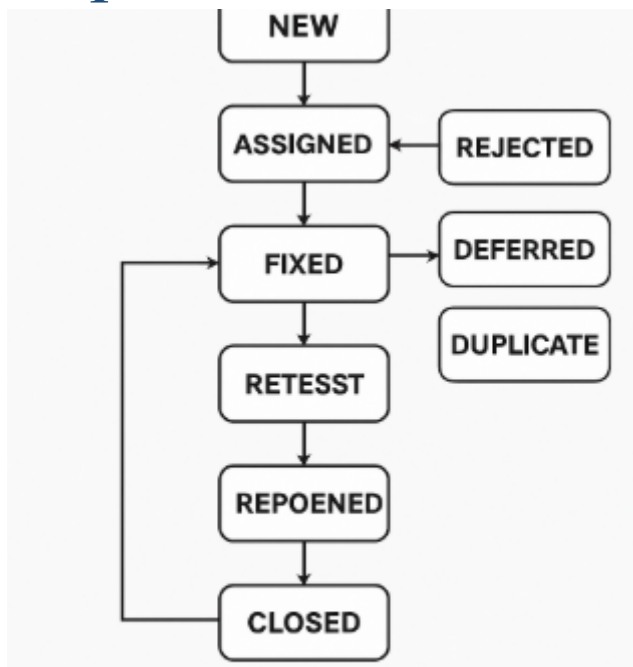
16. Approval:

Name	Role	Signature

[QA Lead Name]	QA Lead	
[PM Name]	Project Manager	
[Client/Stakeholder]	Product Owner	

1. Explain Bug life cycle

Bug Life Cycle (Defect Life Cycle) Explained



The **Bug Life Cycle** refers to the **progression of a defect or bug** through various stages from its identification to its final closure. It helps in tracking and managing bugs efficiently during software testing.

Stages of the Bug Life Cycle

Stage	Description
1. New	When a tester detects a new bug, it is assigned the "New" status and logged in the defect tracking tool.
2. Assigned	The bug is assigned to a developer by the test lead or manager for fixing.
3. Open	The developer has accepted the bug and started analyzing it.
4. In Progress / Fixed	The developer makes the required code changes and marks the bug as “Fixed”.
5. Retest	The bug is passed to the testing team for re-verification.

6. Verified	If the bug no longer exists, the tester marks it as “Verified”.
7. Closed	After verification, the bug is marked as “Closed” if it is resolved.
8. Reopened	If the issue still exists during retesting, it is “Reopened” and reassigned to the developer.
9. Rejected / Not a Bug	The developer may reject the bug if it's invalid, not reproducible, or not considered a bug.
10. Deferred / Postponed	The bug is valid but will be fixed in a later release due to low priority or other constraints.
11. Duplicate	The bug has already been reported earlier. It is marked as “Duplicate”.

Flow Diagram of Bug Life Cycle (Text Representation)

css

CopyEdit

[New]



[Assigned]



[Open]



[Fixed]



[Retest]

→ [Reopened] → [Open] → ...



[Verified]



[Closed]

Tools Used for Tracking Bug Life Cycle:

- . JIRA
- . Bugzilla
- . Mantis

- . Redmine
- . Quality Center (HP ALM)

Why is Bug Life Cycle Important?

- . Ensures **proper tracking** and **communication** between testers and developers.
- . Helps to give **priority** and **resolve** critical issues on time.
- . Improves **quality control** and **customer satisfaction**.

11. What is defect repository? Explain with suitable example.

A Defect Repository (also known as a **Bug Repository** or **Defect Tracking System**) is a

centralized database or system where all the identified defects (bugs) found during software development and testing are **logged, tracked, and managed**.

Purpose of a Defect Repository:

- . To record **details of defects** (e.g., description, status, severity).
- . To **track progress** of each bug through its life cycle.
- . To **analyze trends** for improving product and process quality.
- . To facilitate **communication** between testers, developers, and project managers.

Key Elements in a Defect Repository:

Field	Description
Bug ID	Unique identifier for each defect
Summary	Short description of the

	defect
Description	Detailed explanation of the issue
Severity & Priority	Indicates the impact and urgency
Status	Current state (New, Assigned, Fixed, Closed, etc.)
Reported By	Name of the tester or user who reported the defect
Assigned To	Developer responsible for fixing the issue
Steps to Reproduce	Clear steps to reproduce the issue
Environment	OS, browser, or platform where bug occurred
Attachments	Screenshots, logs, etc.

Example of a Defect Entry in the Repository:

Field	Value
Bug ID	BUG_1035
Summary	Login page crashes on invalid input
Description	App crashes when special characters are entered in username
Severity	High
Priority	Medium
Status	Assigned
Reported By	Tester_Asha
Assigned To	Dev_Rahul
Steps to Reproduce	1. Open login page 2. Enter #!@ in username 3. Click Login

Environment	Windows 11, Chrome 118
Attachments	Screenshot_LoginCrash.png

Tools Used as Defect Repositories:

- . **JIRA**
- . **Bugzilla**
- . **Redmine**
- . **Mantis**
- . **HP ALM (Quality Center)**

12.

Defect Repository	
Bug ID	BUG_1035
Summary	Login page crashes on invalid input
Description	App crashes when special characters are entered in username
Severity	High
Priority	Medium
Status	Assigned
Reported By	Tester_Asha
Assigned To	Dev_Rahul
Steps to Reproduce	1. Open login page 2. Enter "#!@" in username 3. Click Login
Environment	Windows 11, Chrome 118
Attachments	Screenshot_LoginCrash.png

Develop test case for Username and Password

UI Validation for Username and Password

Sl. No.	Input Type	Description	Valid/nvalid	Expected Result
1	Username	Enter username	Valid	Valid username accepted
2	Username	Length less than (<6)	Invalid	Error message occurs. Try valld value
3	Username	Leave blank	Invalid	Error message Mandatory field
4	Password	Enter special characters in username	Invalid	Error message
5	Password	Provide password	Valld	Password accepted
6	Password	Leave blank	Invalid	Error message
7	Password	Special characters in password	Invalid	Password accepted
8	Password	Entr: length of characters agiven (6)	Valid	Error message. Try vofid value
9	Password	Enter characters than specified	Invalid	Error message

fundamental problems oversight, omission.

Verification is a "human" examination or review of the work product.

What is Validation? *Dynamic Execution - user's feel.*

Validation is process of confirming whether software meets users requirements

The process of executing something to see how it behaves.

Unit integration, system, and acceptance testing are examples of validation techniques.

Validation is the process of executing a product to *to find* its defects.

Validation is usually performed by DYNAMIC testing, or testing with execution on a computer.

Eg I've use software A on hardware B and do C then D should happen.
(expected result)

Reviews, Walkthrough, Inspection

White-box testing *checking are we making right problem?*

White box tests require knowledge of the internal program structure and are derived from the internal design specification or the code.

They will not detect missing function (i. e. those described in the functional design specification but not supported by the internal specification or code)

Black-box Testing *Functional, I/O. - George's code of P.*

Are derived from functional design specification, without regard to the internal program structure.

Test the product against the end user, external specifications.

Is done without any internal knowledge of the product.

It will not test hidden functions (i. e. functions, implemented but not described in the functional design specification), and errors associated with them will not be found in black box testing.

Unit Testing *Functional, I/O*

Unit testing is the process of testing the individual components of a program

The purpose is to discover discrepancies between the module's interface specification and its actual behavior.

Typically used to verify the control flow and data flow

It requires the knowledge of the code hence performed by the developers

Integration Testing *Focus on interaction with component.*

Integration testing is the process of combining and testing multiple components together

To assure that the software units / components operate properly when combined together

To discover errors in the interface between the components, Verify Communication between Units

It is done by developers / QA teams

Function Testing

Function testing is the process of attempting to detect *defects difference* discrepancies between a program's functional specification and its actual behavior

It verifies that the software provides expected services *begin to end, correct working properly or not*

Includes positive and negative scenarios i.e. valid inputs and invalid inputs *giving I/O both off.*

System Testing *Before Run*

System testing is the process of attempting to demonstrate that a program or system does not meets its original requirements and objectives, as stated in the requirements specification

Is performed by a testing group before the product is made available to customers.

Functional System testing *check end to end functionality from 2 perspective*

functional - requirements based -> it will 2 knowledge of the business process

It can begin whenever the product has sufficient functionality to execute some of the tests or after unit and integration testing are completed.

The steps of system testing are :

Decompose and analyze the requirements specification

Partition the requirements into logical categories and, for each component, make a list of the detailed requirements

For each type of system testing:

For each relevant requirement, determine inputs and outputs

Develop the requirements test cases

Develop a requirements coverage matrix which is simply a table in which an entry describes a specific subtest that adds value to the requirements coverage, the priority of that subtest, the specific test cases in which that subtest appears

Execute the test cases and measure logic coverage

Develop additional tests, as indicated by the combined coverage information

Types / Goals of System Testing

- Usability testing
- Performance Testing
- Load Testing
- Stress Testing
- Volume Testing
- Security Testing
- Configuration Testing
- Compatibility Testing
- Installability Testing
- Recovery Testing
- Serviceability Testing
- Reliability / availability Testing

Smoke Testing → Smoke testing determines whether the system is sufficiently stable prior to testing the rest of features. also called sanity.

* Exploratory Testing → If you don't have a spec. by using bug-bait, code by - That's not an ideal situation for a QA tester, but you can use a workable technique known as exploratory testing.

Usability Testing : How easy is it for users to bring up what they want

Usability testing is the process of attempting to identify discrepancies between the user interface of a product and the human engineering requirements of its potential users. 3. Like B testing involve users.

Performance testing

The IEEE standard 610.12-1990 (Software Engineering Terminology) defines performance testing : Testing conducted to evaluate the compliance of a system or component with specified performance requirements. (or time) measuring response time, throughput, or resource utilization.

Load Testing

Load testing is subjecting a system to a statistically representative (usually) load.

Stress Testing

The IEEE standard 610.12-1990 (Software Engineering Terminology) defines stress testing : Testing conducted to evaluate a system or component at or beyond the limits of its specified requirements. check beyond limit.

Configuration Testing

To determine whether the program operates properly when the software or hardware is configured in a required manner

Compatibility

To determine whether the compatibility objectives of the program have been met

Installability

To identify the ways in which the installation procedures lead to incorrect results

Installation option

gets corrupt then there should be facility to recover

- New
- Upgrade
- Customized / Complete
- Under normal & abnormal conditions

Important - Makes first impression on the end user

Acceptance Testing

System testing performed by end users to determine whether the software is ready for final deployment

Acceptance testing is the process of comparing the end product to the current needs of its end users.

It is usually performed by the customer or end user after the testing group has satisfactorily completed usability, function, and system testing

It usually involves running and operating the software in production mode for a prespecified period

If the software is developed under contract, acceptance testing is performed by the contracting customer. Acceptance criteria are defined in the contract.

If product is not developed under contract, the developing organization can arrange for alternative forms of acceptance testing

- ALPHA
- BETA

ALPHA and BETA testing are each employed as a form of acceptance testing

Often both are used, in which case BETA follows ALPHA

Both involve running and operating the software in production mode for a pre-specified period.

The ALPHA test is usually performed by end users inside the development organization.

The BETA test is usually performed by a selected subset of actual customers outside the company, before the software is made available to all customers

Regression testing

It is a re-execution of some or all of the tests developed for a specific testing activity for each build of the application

Verify that changes fixes have not introduced new problems.

Equivalence Partitioning

An equivalence class is a subset of data that is representative of a larger class.

Equivalence partitioning is a technique for testing equivalence classes rather than undertaking exhaustive testing of each value of the larger class.

For example, a program which edits credit limits within a given range (\$10,000 - \$ 15,000) would have three equivalence classes :

- Less than \$ 10,000 (invalid)
- Between \$ 10,000 and \$ 15,000 (valid)
- Greater than \$ 15,000 (invalid)

Boundary value analysis

A technique that consists of developing test cases and data that focus on the input and output boundaries of a given function

In same credit limit example, boundary analysis would test:

- Low boundary plus or minus one (\$ 9,999 and \$ 10,001)
- On the boundary (\$10,000 and \$15,000)
- Upper boundary plus or minus one (\$14,999 and \$15,001)

Test case

A set of test inputs, execution conditions, and expected results developed for a

Recovery - e.g. ms word
must for database servers as data
is important.
Security - such as password, level of
authority.

particular objective.

The smallest entity that is always executed as unit, from beginning to end. A test case is a document that describes an input, action, or event and an expected response, to determine if a feature of an application is working correctly. A test case should contain particulars such as test case identifier, test case name, objective, test conditions / setup, input data requirements, steps, and expected results.

Test case may also include prerequisites.

A 'test case' is another name for "scenario". It is a particular situation to be tested, the objective or goal of the test script. For example in an ATM banking application, a typical scenario would be: "Customer deposits cheque for \$1000 to an account that has a balance of \$150, and then attempts to withdraw \$200". Every Test Case has a goal; that is, the function to be tested. Quite often the goal will begin with the words "To verify that..." and the rest of the goal is a straight copy of the functional test defined in the Traceability Matrix. In the banking example above, the goal might be worded as "To verify that error message 63 ('Insufficient cleared funds available') is displayed when the customer deposits a cheque for \$1000 to an account with a balance of \$150, and then attempts to withdraw \$ 200".

The structure of test cases is one of the things that stays remarkably the same regardless of the technology being tested. The conditions to be tested may differ greatly from one technology to the next, but you still need to know three basic things about what you plan to test :

ID# This is a unique identifier for the test case. The identifier does not imply a Sequential order of test execution in most cases. The test case ID can also be intelligent.

For example, the test case ID of ORD001 could indicate a test case for the ordering process on the first web page.

Condition: This is an event that should produce an observable result. For example, in an e-commerce application, if the user selects an overnight shipping option, the correct charge should be added to the total of the transaction. A test designer would want to test all shipping options, with each option giving a different amount added to the transaction total.

Expected Result: This is the observable result from invoking a test condition. If you can't observe a result, you can't determine if a test passes or fails. In the previous example of an e-commerce shipping option, the expected results would be specifically defined according to the type of shipping the user selects.

Procedure : This is the process a tester needs to perform to invoke the condition and observe the results. A test case procedure should be limited to the steps needed to perform a single test case.

Pass/Fail : This is where the tester indicates the outcome of the test case. For the purpose of space, I typically use the same column to indicate both "pass" (P) and "fail" (F). In some situations, such as the regulated environment, simply indicating pass or fail is not enough information about the outcome of a test case to provide adequate documentation. For this reason, some people choose to also add a column for "observed Results".

Defect Number Cross-reference : If you identify a defect in the execution of a test case, this component of the test case gives you a way to link the test case to a specific defect report.

What is definition of defect ?

A flaw in a system or system component that causes the system or component to

fail to perform its required function.
Some typical defect report fields

Summary
Date reported
Detailed description
Assigned to
Severity
Detected in Version
Priority
System info
Status
Reproducible
Detected by
Screen prints, logs, etc.

Traceability Matrix

Traceability Matrix relates each functional requirement to a specific test case. Sometimes there is one test case for each requirement; other times, several requirements can be validated by one longer test case.

State Testing - A s/w state is a condition or mode that the s/w is currently in. s/w tester must test a program's states & the transitions between them.

e.g. In paintbrush app, pencil drawing state

Volume - To determine the program can handle the req'd volume of data, request etc

Exploratory - If you don't have any Req^{ts} or specifications then we have to learn & check appⁿ is called Exploratory testing

Ad-hoc & Random Testing - Giving Randomly "I/P to the appⁿ" is called

Ad-hoc & Random Testing
Monkey - Just giving any comb of I/P & data

Mutation - Keep changing

Localization - User
Process of Adopting s/w to a specific locality talking into Accents in a lang. When local convention & culture is called localization testing

Reliability - Prob. of failure of system for a specific time or specific period of time in a specified environment.

$MTBF = MTF + MTTR$ (approx)

Availability - Prob that a program operates successfully to the user at a given pt. of time.

$Availability = \frac{MTF}{MTF + MTTR} \times 100\%$

Releasing - Because any s/w product that is actively used & supported must be changed from time to time. Every new version of a product should be released.

Regression & confirmation - confirmation testing ensures that defect is fixed but regression testing ensures that defect is not introduced.

ATM TEST CASE

1. Functional Test Cases

Test Case ID	Description	Steps	Expected Result
TC01	Verify card insertion	Insert valid ATM card	Card is read successfully
TC02	Invalid card detection	Insert expired or damaged card	"Invalid Card" error shown
TC03	Valid PIN entry	Enter correct 4-digit PIN	Access granted
TC04	Invalid PIN entry	Enter wrong PIN 3 times	Card is blocked, error shown
TC05	Balance inquiry	Select "Balance Inquiry" after login	Display correct account balance
TC06	Cash withdrawal	Select "Withdraw", enter amount ₹500	Dispense ₹500 if balance sufficient
TC07	Insufficient balance	Withdraw more than available balance	Show "Insufficient funds"
TC08	Cash deposit	Select "Deposit", insert envelope or cash	Update balance, show confirmation
TC09	Mini-statement	Request mini-statement	Print last 5–10 transactions
TC10	PIN change	Select "Change PIN", enter new PIN twice	Update PIN and show success

✓ 2. UI Test Cases

Test Case ID	Description	Steps	Expected Result
TC11	Verify main	Power on ATM	Buttons, language

	screen layout		selection, card slot clearly visible
TC12	Language change	Select "Marathi" or "Hindi"	Interface language updates correctly
TC13	Touchscreen response	Tap each option	Fast, accurate touch response
TC14	Error message clarity	Enter invalid PIN	Clear, readable error message shown
TC15	Cancel operation	Press "Cancel" mid-transaction	Return to welcome screen

✓ 3. Boundary & Limit Test Cases

Test Case ID	Description	Steps	Expected Result
TC16	Max withdrawal limit	Try to withdraw ₹25,000 (if limit is ₹20,000)	Show "Limit Exceeded"
TC17	Minimum amount	Try to withdraw ₹50 (if min is ₹100)	Show "Below Minimum Limit"

TC18	Card session timeout	Insert card and stay idle	Auto-cancel transaction after timeout
TC19	PIN length validation	Enter 3-digit or 5-digit PIN	Show "Invalid PIN format"
TC20	Daily transaction count	Perform more than allowed number of daily transactions	Show "Daily Limit Reached"

✓ 4. Security Test Cases

Test Case ID	Description	Steps	Expected Result
TC21	Shoulder-surfing prevention	Observe ATM usage from behind	PIN is masked (****)
TC22	Session termination	Do not press "Exit" after transaction	Session auto-ends after timeout
TC23	Skimming detection	Check for unauthorized devices	Alert raised if suspicious device found
TC24	Card trapping attempt	Try to pull back card forcefully	ATM retracts card, locks it

TC25	Power failure	Turn off power during transaction	ATM rolls back incomplete transaction
------	---------------	-----------------------------------	---------------------------------------

✓ 5. Integration Test Cases

Test Case ID	Description	Steps	Expected Result
TC26	Bank database sync	Withdraw ₹1000 from ATM	Amount debited in real-time from account
TC27	Notification system	Complete withdrawal	SMS/email sent with details
TC28	Inter-bank transaction	Use different bank's card	ATM processes request via network
TC29	Currency mismatch	Withdraw from foreign account	Currency conversion shown
TC30	Receipt printing	Complete any transaction	Receipt printed with correct data

PASSWORD TEST CASE

1. Functional Test Cases

Test Case ID	Description	Test Steps	Expected Result
TC01	Verify correct password	Enter valid password and submit	Login successful
TC02	Verify incorrect password	Enter wrong password and submit	Error message: "Invalid password"
TC03	Empty password field	Leave password field blank and submit	Error: "Password is required"
TC04	Case sensitivity check	Enter same password in different case (e.g., "Password" vs "password")	Treat as different passwords
TC05	Password with special characters	Enter password like "Pa\$\$w0rd!"	Accepted if valid

TC06	Password with spaces	Enter password with leading/trailing spaces	Spaces either trimmed or error shown
------	----------------------	---	--------------------------------------

✓ 2. Boundary & Limit Test Cases

Test Case ID	Description	Test Steps	Expected Result
TC07	Minimum length password	Enter password with exactly minimum characters (e.g., 6)	Accepted
TC08	Below minimum length	Enter 5 characters (if min is 6)	Error: "Password too short"
TC09	Maximum length password	Enter password with max allowed length (e.g., 20)	Accepted
TC10	Above maximum length	Enter 21 characters	Error: "Password too long"
TC11	Password with only numbers	Enter "12345678"	Accepted/rejected per policy
TC12	Password with only letters	Enter "abcdefgh"	Accepted/rejected per policy

✓ 3. UI/UX Test Cases

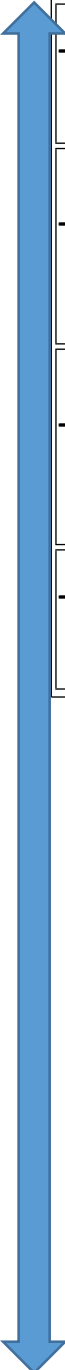
Test Case ID	Description	Test Steps	Expected Result
TC13	Password masking	Enter password	Characters appear as *** or •
TC14	Show/hide toggle	Click "eye" icon	Password becomes visible/invisible
TC15	Password field focused	Click into password field	Cursor appears, field highlights
TC16	Paste password	Copy and paste password into field	Paste accepted or blocked as per requirement

✓ 4. Security Test Cases

Test Case ID	Description	Test Steps	Expected Result
TC17	Brute-force prevention	Try wrong password 5 times	Account locked or CAPTCHA shown
TC18	SQL Injection in password	Enter ' OR '1'='1	Input sanitized, error shown
TC19	Clipboard clearing	Paste password and navigate away	Password not retained in clipboard
TC20	Session timeout	Enter password but stay idle	Session expires, user logged out

✓ 5. Password Strength Test Cases (For Signup Forms)

Test Case	Description	Test Steps	Expected Result
-----------	-------------	------------	-----------------

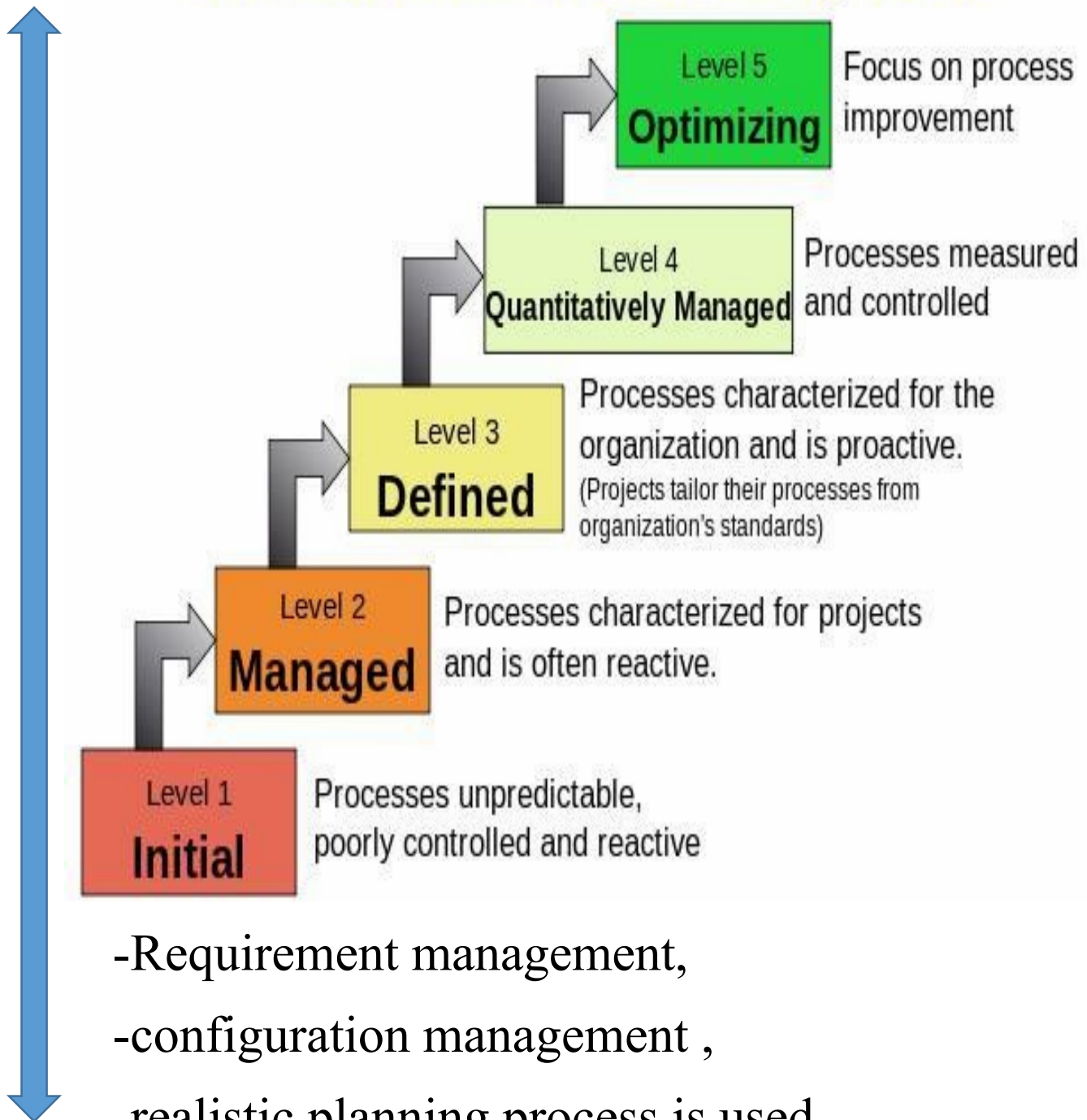


ID			
TC21	Weak password	Enter "123456"	Show "Weak Password" warning
TC22	Strong password	Enter "Xyz@1234\$"	Show "Strong Password" message
TC23	Password policy enforcement	Enter password missing uppercase	Error: "Must include uppercase letter"
TC24	Confirm password mismatch	Enter different values in Password and Confirm Password	Error: "Passwords do not match"
TC25	Password reuse restriction	Try old password	Show: "Password already used"

SEI/CMM

SOFTWARE ENGG INSTITUTE/CAPABILITY MATURITY MODEL(NOW CALLED CMMI, CAPABILITY MATURITY MODEL INTEGRATION)

Characteristics of the Maturity levels



- Requirement management,
- configuration management ,
- realistic planning process is used.
- Basic project management processes are established, and successes can be repeated.

Level 3 : DEFINED

- Standard development and maintenance process are used in organization
- Training programs are used to ensure understanding and compliance
- Standard software development processes are defined and documented.

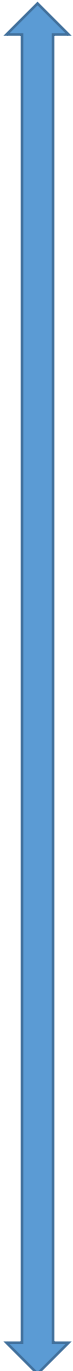
Level 4: QUANTITATIVELY MANAGED

- Matrices are used to track productivity
- also used processes and products
- performance is predictable
- Quality is consistently high
- Processes are monitored and controlled through data collection and analysis.

Level 5: OPTIMIZED

- 
- The focus is on continuous improvement.
 - New process and technologies can be effectively implemented when require.
 - Processes are constantly improved through feedback and innovation.

Bug



Defination: A bug in software refers to an error, flaw, or fault that causes a program to produce an incorrect or unexpected result, or behave in unintended ways.

The Bug Life Cycle is a structured process that tracks a bug from its initial discovery to its resolution, ensuring efficient bug management and improved software quality.

Bug Types:

Bugs can be classified based on various factors, including:

- **Severity:**

How severely the bug impacts the software's functionality (e.g., critical, major, minor, trivial).

- **Priority:**

How urgent it is to fix the bug (e.g., high, medium, low).

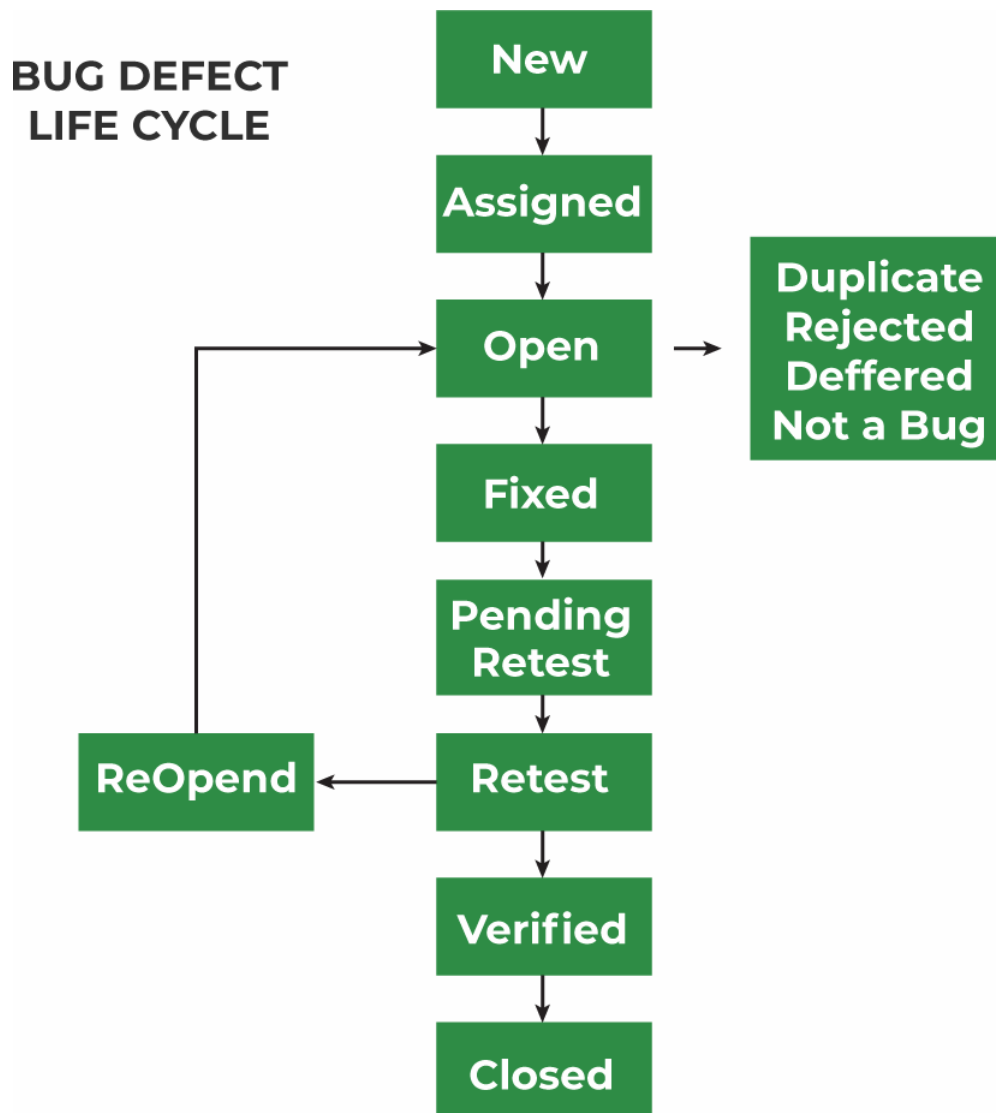
• **Reproducible:**

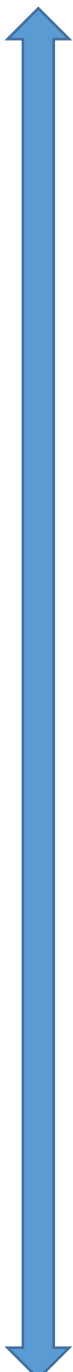
Whether the bug can be consistently reproduced by different users or testers.

• **Scope:**

Whether the bug is localized to a specific module or affects the entire application.

Bug Life Cycle Stages:





The bug life cycle typically involves the following stages:

1. New

- Whenever a new defect is registered and posted for the first time, a “NEW” status is assigned to it.
- The tester offers a detailed defect/bug report to the Development team for resolving bug.

2. Assigned

- After the tester posts the bug, the tester’s lead authorises the bug and assigns it to the developer team.
- In this stage, there can be two conditions. Firstly, the defect can be directly assigned to the developer.
- Secondly, it can be assigned to the Dev Lead and after being approved by the Dev Lead, they can further transfer the defect to the developer.

3. Open

-In this stage, the developer begins to analyse and resolve the defect.

- If the developer team perceives that the defect is inappropriate, it is moved to either the 'Deferred' or the 'Rejected' state.

4. Fixed

-A developer assigns a "Fixed" status to a bug after making a relevant code change and verifying the change.

5. Test

-The bug status as "Test" indicates that the defect is fixed and is ready to perform the test.

-The test is done to verify that the bug is fixed.

6. Verified

- In this stage, the tester re-tests the bug after being fixed by the developer.
- In case the bug is not reproduced, the bug is considered fixed, and a “verified” status is assigned to it.

7. Closed

- If the bug is resolved, the tester allocates the “Closed” status to it.
- After the bug is fixed, the tester tests it. This stage indicates that the bug is fixed, tested, and validated.

8. Reopen

- In certain scenarios, the bug exists even after the developer has resolved it.
- In those cases, the tester assigns a “Reopen” state to the bug.

Here, the bug passes through the same life cycle again after the “Reopen” bug is moved to “fixed” state.

9. Duplicate

Sometime, the same bug may be reported twice. In that case, the bug status is modified to “duplicate.” and, the defect is rejected.

10. Deferred

The tester team assigns a “Deferred” state to bugs in the following cases:

- If the bug is not of key priority and is expected to be resolved in the forthcoming release.

- The customer will modify the requirement

Note that the high-priority bugs can't and should not be deferred.



11. Rejected

-If the developer perceives that the defect is due to *misinterpretation or is not genuine*, they will assign a “Rejected” state to the defect. Usually, the reason for rejection may be one of the following:

NOT a Defect

Duplicate Defect

Non-Reproducible

Severity vs Priority

Bug vs Defect vs Error vs Fault vs Failure

1.

Bug

A bug refers to defects which means that the software product or the application is not working as per the adhered requirements set

Raised by

Test Engineers

- Different types
 - Logical bugs
 - Algorithmic bugs
 - Resource bugs
- Reasons behind
 - Missing Logic
 - Erroneous Logic
 - Redundant codes

Defect

A Defect is a deviation between the actual and expected output

The defect is identified by The Testers And is resolved by developers in the development phase of SDLC.

- **Different types**

Defects are classified as follows:

Based on Priority:

- High
- Medium
- Low

Based on Severity:

- Critical
- Major
- Minor
- Trivial

- **Reasons behind**

- Receiving & providing incorrect input

Defect

- . Coding/Logical Error leading to the breakdown of software

- . Way to prevent
- . Implementing Out-of-the-box programming methods.
- . Proper usage of primary and correct software coding practices.

Fault

A Fault is a state that causes the software to fail and therefore it does not achieve its necessary function.

Human mistakes lead to fault.

- . Types

Fault

- . Business Logic Faults
 - . Functional and Logical Faults
 - . Graphical User Interface (GUI) Faults
 - . Performance Faults
 - . Security Faults
 - . Hardware Faults
-
- . Reason behind
 - . Wrong design of the data definition processes.
 - . An irregularity in Logic or gaps in the software leads to the non-functioning of the software.

Error

An Error is a mistake made in the code due to which compilation or execution fails,

Developers and automation test engineers

- . Types
 - . Syntactic Error
 - . UI screen error
 - . Error handling error
 - . Flow control error
 - . Calculation error
 - . Hardware error
- . Reason behind:
 - . Error in code.
 - . Inability to compile/execute a program
 - . Misunderstanding of requirements
 - . Faulty design and architecture
 - . Logical error

Failure

Failure is the accumulation of several defects that ultimately lead to Software failure and results in the loss of information

The failure is found by the test engineer during the development cycle of SDLC

Types:

NA

- . Reason:
- . Environment variables
- . System Errors
- . Human Error

TEST PLAN TEMPLATE

Test Plan Template:

(Name of the Product)

Prepared by:

(Names of Preparers)

(Date)

TABLE OF CONTENTS

1.0 INTRODUCTION

2.1 OBJECTIVES AND TASKS

2.1 Objectives

2.2 Tasks

3.0 SCOPE

4.1 Testing Strategy

4.1 Alpha Testing (Unit Testing)

4.2 System and Integration Testing

4.3 Performance and Stress Testing

4.4 User Acceptance Testing

4.5 Batch Testing

4.6 Automated Regression Testing

4.7 Beta Testing

5.0 Hardware Requirements

6.1 Environment Requirements

6.1 Main Frame

6.2 Workstation

7.0 Test Schedule

8.0 Control Procedures

9.0 Features to Be Tested

10.0 Features Not to Be Tested

The seven principles of software testing are:

- 1) Testing shows the presence of defects, not their absence. 2
- 2) Exhaustive testing is impossible.
- 3) Early testing saves time and money.
- 4) Defects cluster together.
- 5) Beware the pesticide paradox.
- 6) Testing is context-dependent.
- 7) The absence of errors is a fallacy.

Here's a more detailed explanation:

1. Testing shows the presence of defects, not their absence:

Testing can find defects, but it cannot prove that a system is completely defect-free.

2. Exhaustive testing is impossible:

It's not feasible to test all possible combinations of inputs, preconditions, and scenarios.

3. Early testing saves time and money:

Starting testing early in the development lifecycle helps identify and fix defects sooner, reducing costs.

4. Defects cluster together:

A small number of modules or components often contain the majority of defects.

5. Beware the pesticide paradox:

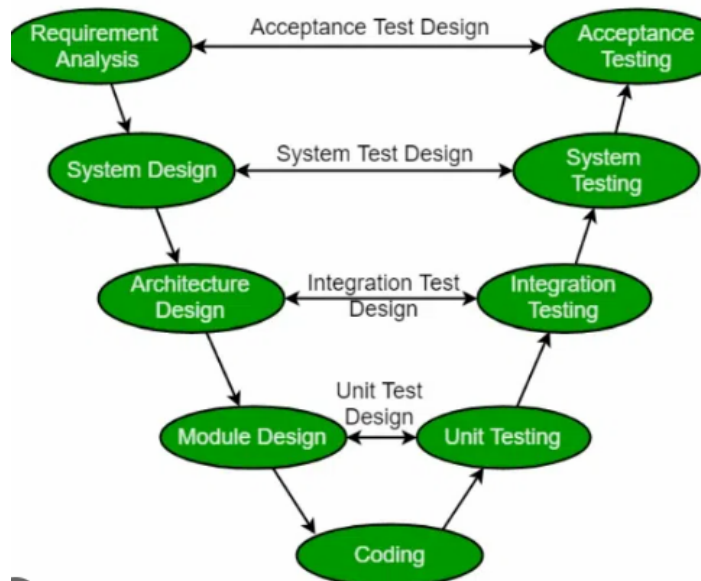
Running the same tests repeatedly will eventually fail to find new defects. Test cases need to be updated and revised.

6. Testing is context-dependent:

The approach to testing should be tailored to the specific context of the project, including the application type, risk level, and user requirements.

1. 7. The absence of errors is a fallacy:

A system can be technically defect-free but still not meet user needs or be unusable.



The V-model in software testing is a Software Development Life Cycle (SDLC) model that emphasizes verification and validation, visually represented as a "V" shape. It links each development phase with a corresponding testing phase, ensuring rigorous testing at each stage. The left side of the "V" represents development phases (requirements, design, etc.), while the right side mirrors those with testing phases (unit testing, integration testing, etc.).

Development (Left Side of the V):

- **Requirements Analysis:** Defining the system's overall functionality and features.
- **System Design:** High-level design of the system architecture and components.
- **Architectural Design:** Detailed design of specific modules and their interactions.

- **Module Design:** Designing individual modules and their internal logic.
 - **Coding:** Actual programming of the software modules.
- Testing (Right Side of the V):
- **Unit Testing:** Testing individual components or modules in isolation.
 - **Integration Testing:** Testing the interaction between different modules.
 - **System Testing:** Testing the entire system against the specified requirements.
 - **Acceptance Testing:** Testing by the end-user to ensure the system meets their needs.

Black Box Testing Vs White Box Testing

Parameters	Black Box Testing	White Box Testing
Definition	Black Box Testing is a way of software testing in which the internal structure or the program or the code is hidden and nothing is known about it.	White Box Testing is a way of testing the software in which the tester has knowledge about the internal structure or the code or the program of the software.
Testing objectives	Black box testing is mainly focused on testing the functionality of the software, ensuring that it meets the requirements and specifications.	White box testing is mainly focused on ensuring that the internal code of the software is correct and efficient.
Testing methods	Black box testing uses methods like <u>Equivalence Partitioning</u> , <u>Boundary Value Analysis</u> ,	White box testing uses methods like <u>Control Flow Testing</u> , <u>Data Flow Testing</u> and <u>Statement</u>

	and <u>Error Guessing</u> to create test cases.	<u>Coverage Testing</u> .
Knowledge level	Black box testing does not require any knowledge of the internal workings of the software, and can be performed by testers who are not familiar with programming languages.	White box testing requires knowledge of programming languages, software architecture and design patterns.
Scope	Black box testing is generally used for testing the software at the functional level.	White box testing is used for testing the software at the unit level, integration level and system level.
Implementation	Implementation of code is not needed for black box testing.	Code implementation is necessary for white box testing.
Done By	Black Box Testing is mostly done by software testers.	White Box Testing is mostly done by software developers.
Terminology	Black Box Testing can be referred to as outer or external software testing.	White Box Testing is the inner or the internal software testing.
Testing Level	Black Box Testing is a functional test of the software.	White Box Testing is a structural test of the software.
Testing Initiation	Black Box testing can be initiated based on the requirement specifications document.	White Box testing of software is started after a detail design document.
Programming	No knowledge of programming is required.	It is mandatory to have knowledge of

		programming.
Testing Focus	Black Box Testing is the behavior testing of the software.	White Box Testing is the logic testing of the software.
Applicability	Black Box Testing is applicable to the higher levels of testing of software.	White Box Testing is generally applicable to the lower levels of software testing.
Alternative Names	Black Box Testing is also called closed testing.	White Box Testing is also called as clear box testing.
Time Consumption	Black Box Testing is least time consuming.	White Box Testing is most time consuming.
Suitable for Algorithm Testing	Black Box Testing is not suitable or preferred for algorithm testing.	White Box Testing is suitable for algorithm testing.
Approach	Can be done by trial and error ways and methods.	Data domains along with inner or internal boundaries can be better tested.
Example	Search something on google by using keywords	By input to check and verify loops
Exhaustiveness	It is less exhaustive as compared to white box testing.	It is comparatively more exhaustive than black box testing.

1.1 INTRODUCTION

A brief summary of the product being tested. Outline all the functions at a high level.

2.1 OBJECTIVES AND TASKS

2.2 Objectives

Describe the objectives supported by the Master Test Plan, eg., defining tasks and responsibilities, vehicle for communication, document to be used as a service level agreement, etc.

2.2 Tasks

List all tasks identified by this Test Plan, i.e., testing, post-testing, problem reporting, etc.

3.0 SCOPE

General

This section describes what is being tested, such as all the functions of a specific product, its existing interfaces, integration of all functions.

Tactics

List here how you will accomplish the items that you have listed in the "Scope" section. For example, if you have mentioned that you will be testing the existing interfaces, what would be the procedures you would follow to notify the key people to represent their respective areas, as well as allotting time in their schedule for assisting you in accomplishing your activity?

4.0 TESTING STRATEGY

Describe the overall approach to testing. For each major group of features or feature combinations, specify the approach which will ensure that these feature groups are

adequately tested. Specify the major activities, techniques, and tools which are used to test the designated groups of features.

The approach should be described in sufficient detail to permit identification of the major testing tasks and estimation of the time required to do each one.

4.1 Unit Testing

Definition:

Specify the minimum degree of comprehensiveness desired. Identify the techniques which will be used to judge the comprehensiveness of the testing effort (for example, determining which statements have been executed at least once). Specify any additional completion criteria (for example, error frequency). The techniques to be used to trace requirements should be specified.

Participants:

List the names of individuals/departments who would be responsible for Unit Testing.

Methodology:

Describe how unit testing will be conducted. Who will write the test scripts for the unit testing, what would be the sequence of events of Unit Testing and how will the testing activity take place?

4.2 System and Integration Testing

Definition:

List what is your understanding of System and Integration Testing for your project.

Participants:

Who will be conducting System and Integration Testing on your project? List the individuals that will be responsible for this activity.

Methodology:

Describe how System & Integration testing will be conducted. Who will write the test scripts for the unit testing, what would be sequence of events of System & Integration Testing, and how will the testing activity take place?

4.3 Performance and Stress Testing

Definition:

List what is your understanding of Stress Testing for your project.

Participants:

Who will be conducting Stress Testing on your project? List the individuals that will be responsible for this activity.

Methodology:

Describe how Performance & Stress testing will be conducted. Who will write the test scripts for the testing, what would be sequence of events of Performance & Stress Testing, and how will the testing activity take place?

4.4 User Acceptance Testing**Definition:**

The purpose of acceptance test is to confirm that the system is ready for operational use. During acceptance test, end-users (customers) of the system compare the system to its initial requirements.

Participants:

Who will be responsible for User Acceptance Testing? List the individuals' names and responsibility.

Methodology:

Describe how the User Acceptance testing will be conducted. Who will write the test scripts for the testing, what would be sequence of events of User Acceptance Testing, and how will the testing activity take place?

4.5 Batch Testing**4.6 Automated Regression Testing****Definition:**

Regression testing is the selective retesting of a system or component to verify that modifications have not caused unintended effects and that the system or component still works as specified in the requirements.

Participants:**Methodology:****4.7 Beta Testing****Participants:****Methodology:****5.0 HARDWARE REQUIREMENTS**

Computers
Modems

6.1 ENVIRONMENT REQUIREMENTS**6.2 Main Frame**

Specify both the necessary and desired properties of the test environment. The

specification should contain the physical characteristics of the facilities, including the hardware, the communications and system software, the mode of usage (for example, stand-alone), and any other software or supplies needed to support the test. Also specify the level of security which must be provided for the test facility, system software, and proprietary components such as software, data, and hardware.

Identify special test tools needed. Identify any other testing needs (for example, publications or office space). Identify the source of all needs which are not currently available to your group.

6.2 Workstation

7.0 TEST SCHEDULE

Include test milestones identified in the Software Project Schedule as well as all item transmittal events.

Define any additional test milestones needed. Estimate the time required to do each testing task. Specify the schedule for each testing task and test milestone. For each testing resource (that is, facilities, tools, and staff), specify its periods of use.

8.0 CONTROL PROCEDURES

Problem Reporting

Document the procedures to follow when an incident is encountered during the testing process. If a standard form is going to be used, attach a blank copy as an "Appendix" to the Test Plan. In the event you are using an automated incident logging system, write those procedures in this section.

Change Requests

Document the process of modifications to the software. Identify who will sign off on the changes and what would be the criteria for including the changes to the current product. If the changes will affect existing programs, these modules need to be identified.

9.0 FEATURES TO BE TESTED

Identify all software features and combinations of software features that will be tested.

10.0 FEATURES NOT TO BE TESTED

Identify all features and significant combinations of features which will not be tested and the reasons.

11.0 RESOURCES/ROLES & RESPONSIBILITIES

Specify the staff members who are involved in the test project and what their roles are going to be (for example, Mary Brown (User) compile Test Cases for Acceptance Testing). Identify groups responsible for managing, designing, preparing, executing, and resolving the test activities as well as related issues. Also identify groups responsible for providing the test environment. These groups may include developers, testers, operations staff, testing services, etc.

12.0 SCHEDULES

Major Deliverables

Identify the deliverable documents. You can list the following documents:

- Test Plan
- Test Cases
- Test Incident Reports
- Test Summary Reports

13.0 SIGNIFICANTLY IMPACTED DEPARTMENTS (SIDs)

Department/Business Area	Bus. Manager	Tester(s)
--------------------------	--------------	-----------

14.0 DEPENDENCIES

Identify significant constraints on testing, such as test-item availability, testing-resource availability, and deadlines.

15.0 RISKS/ASSUMPTIONS

Identify the high-risk assumptions of the test plan. Specify contingency plans for each (for example, delay in delivery of test items might require increased night shift scheduling to meet the delivery date).

16.0 TOOLS

List the Automation tools you are going to use. List also the Bug tracking tool here.

17.0 APPROVALS

Specify the names and titles of all persons who must approve this plan. Provide space for the signatures and dates.

Name (In Capital Letters)	Signature	Date
---------------------------	-----------	------

1.		
----	--	--

2.		
----	--	--

3.		
----	--	--

4.		
----	--	--

End.		
------	--	--